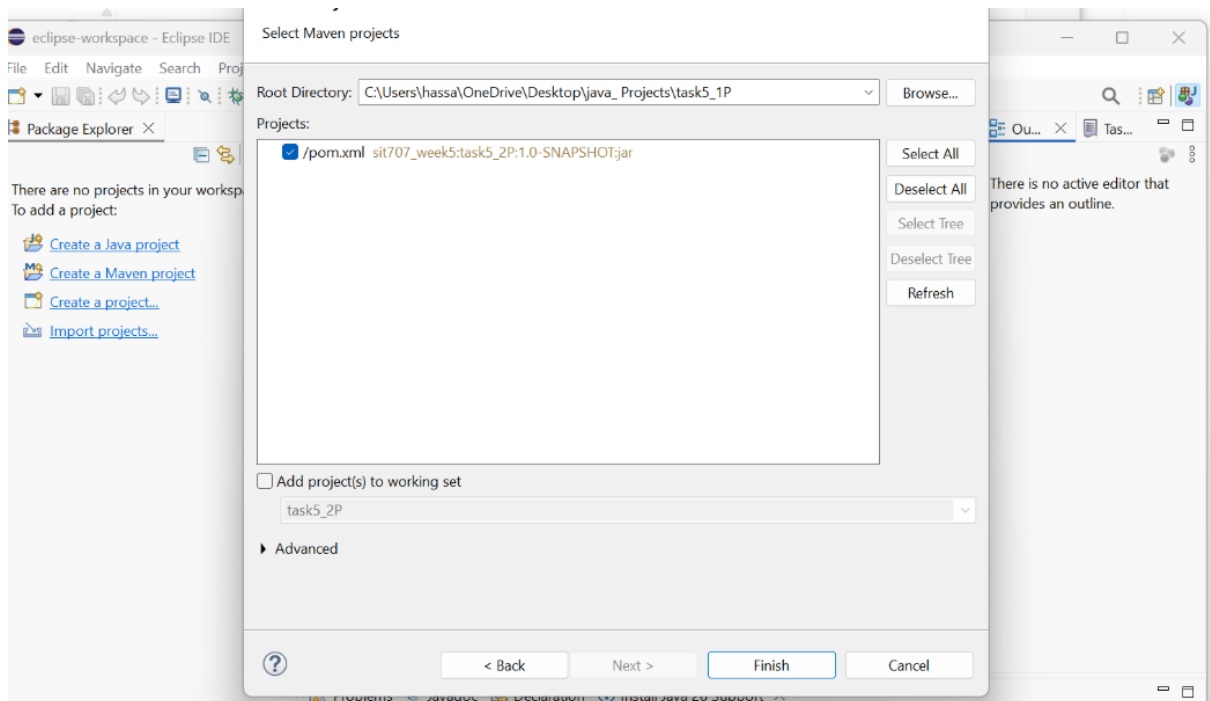
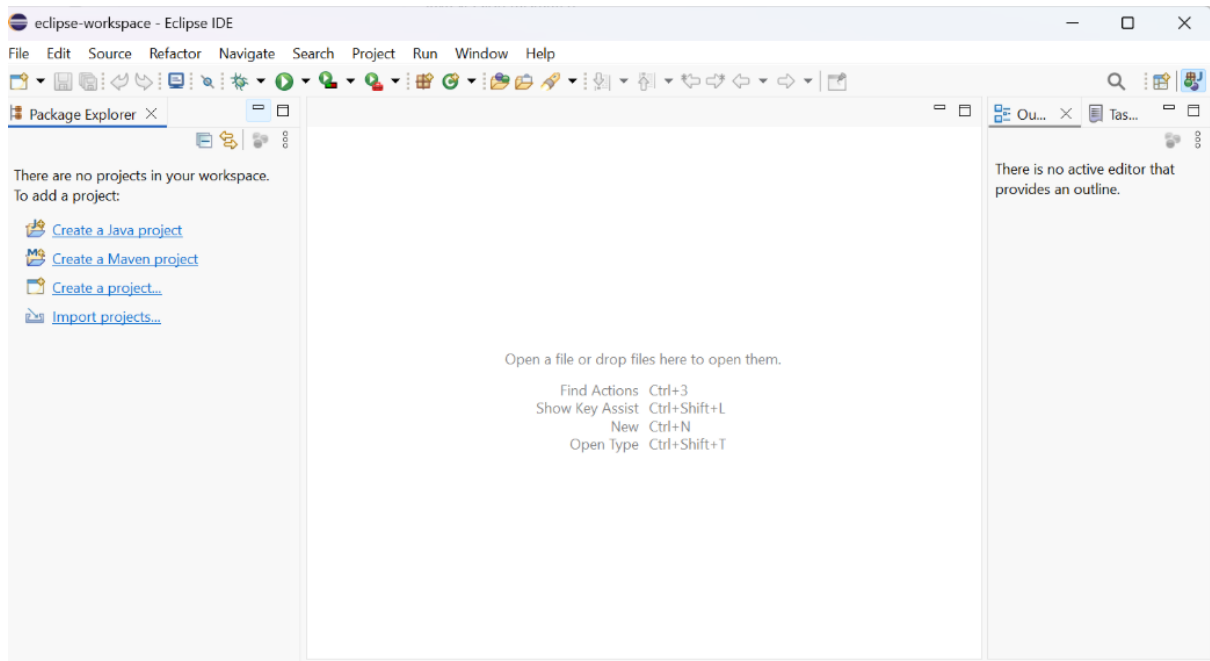
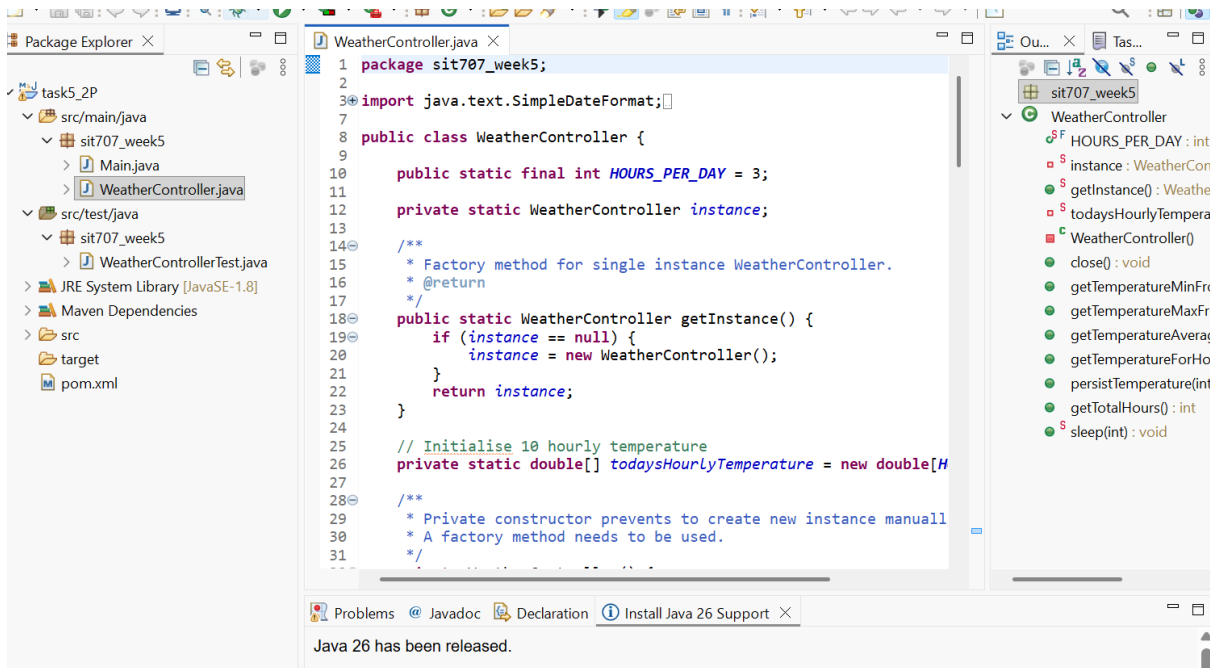


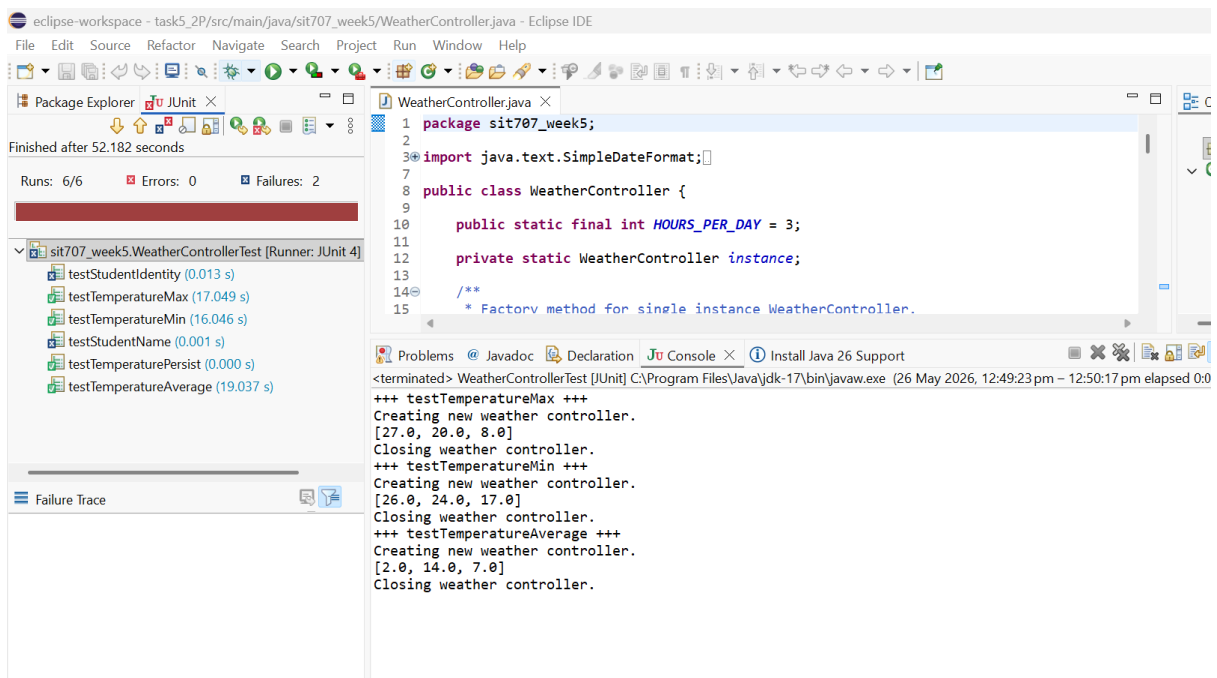
Pass Task 5: Unit test standards – AAA and [F]IRST

Step 1 & 2

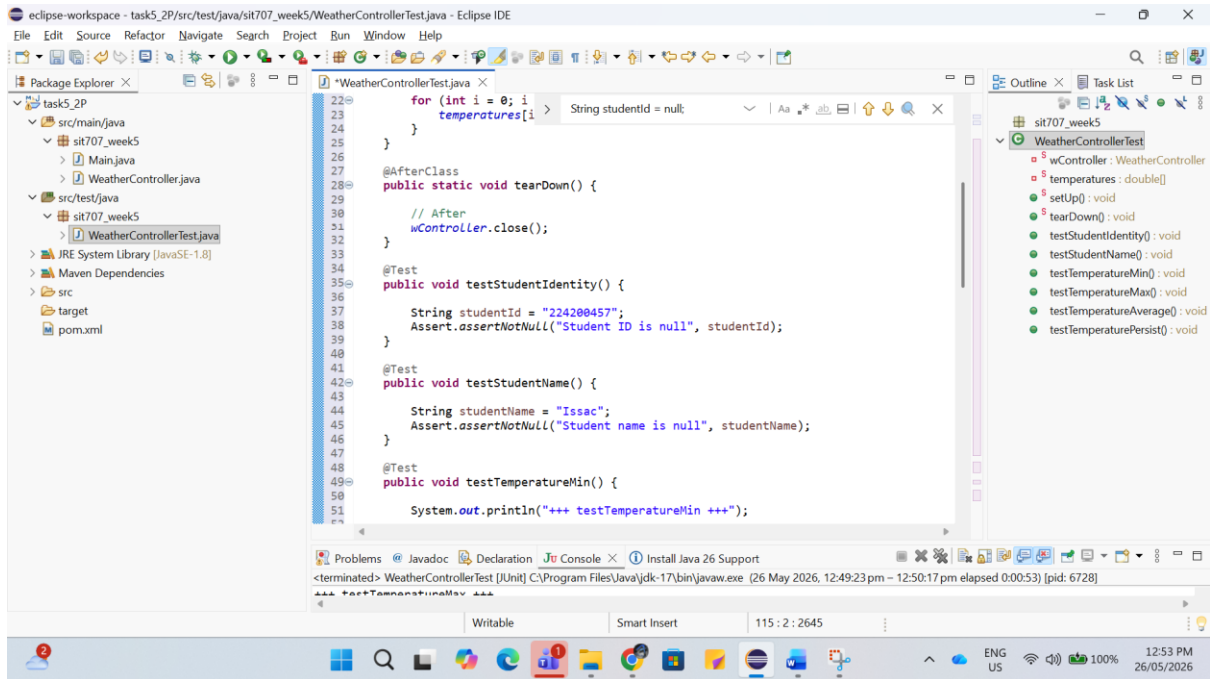




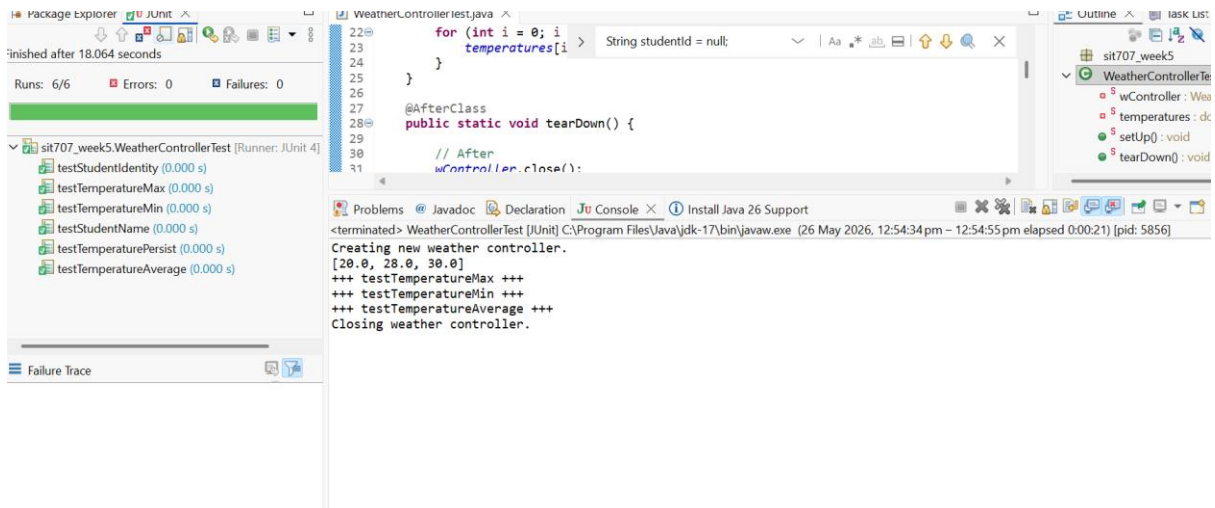
1. Run original test first



2. Fix student ID and name



Console output



Source code:

```
package sit707_week5;
```

```
import org.junit.AfterClass;
```

```
import org.junit.Assert;
```

```
import org.junit.BeforeClass;
```

```
import org.junit.Test;
```

```
public class WeatherControllerTest {
```

```
    private static WeatherController wController;
```

```
    private static double[] temperatures;
```

```
@BeforeClass
```

```
    public static void setUp() {
```

```
        // Arrange
```

```
        wController = WeatherController.getInstance();
```

```
        int nHours = wController.getTotalHours();
```

```
        temperatures = new double[nHours];
```

```
        for (int i = 0; i < nHours; i++) {
```

```
            temperatures[i] = wController.getTemperatureForHour(i + 1);
```

```
        }
```

```
    }
```

```
@AfterClass
```

```
    public static void tearDown() {
```

```
// After  
  
wController.close();  
  
}
```

```
@Test  
  
public void testStudentIdentity() {  
  
    String studentId = "224200457";  
    Assert.assertNotNull("Student ID is null", studentId);  
  
}
```

```
@Test  
  
public void testStudentName() {  
  
    String studentName = "Issac";  
    Assert.assertNotNull("Student name is null", studentName);  
  
}
```

```
@Test  
  
public void testTemperatureMin() {  
  
    System.out.println("+++ testTemperatureMin +++");  
  

```

```
// Arrange  
  
double minTemperature = 1000;
```

```
// Act  
  
for (double temperatureVal : temperatures) {  
    if (minTemperature > temperatureVal) {
```

```

        minTemperature = temperatureVal;
    }
}

// Assert
Assert.assertEquals(minTemperature,
    wController.getTemperatureMinFromCache(), 0.001);
}

@Test
public void testTemperatureMax() {

    System.out.println("+++ testTemperatureMax +++");

    // Arrange
    double maxTemperature = -1;

    // Act
    for (double temperatureVal : temperatures) {
        if (maxTemperature < temperatureVal) {
            maxTemperature = temperatureVal;
        }
    }

    // Assert
    Assert.assertEquals(maxTemperature,
        wController.getTemperatureMaxFromCache(), 0.001);
}

```

@Test

public void testTemperatureAverage() {

System.**out**.println("+++ testTemperatureAverage +++");

// Arrange

double sumTemp = 0;

// Act

for (**double** temperatureVal : *temperatures*) {

sumTemp += temperatureVal;

}

double averageTemp = sumTemp / *temperatures*.length;

// Assert

Assert.*assertEquals*(averageTemp,

wController.getTemperatureAverageFromCache(), 0.001);

}

@Test

public void testTemperaturePersist() {

/*

* Remove below comments ONLY for 5.3C task.

*/

}

}

Short explanation: AAA and FIRST

AAA means each unit test should be organised into clear stages: Arrange, Act, Assert, and sometimes After. In this task, the setup code creates the WeatherController once, retrieves the hourly temperatures once, then each test only performs its own calculation and assertion. This follows the task requirement to restructure the tests, so they run faster instead of repeatedly calling slow methods such as `getInstance()`, `close()`, and `getTemperatureForHour()` .

FIRST means good unit tests should be Fast, Isolated, Repeatable, Self-validating, and Timely. The improved test class mainly focuses on the Fast property by avoiding repeated slow setup and repeated temperature queries. It also improves Self-validating because the JUnit assertions clearly show pass or fail results in the JUnit tab.

Use cases where unit testing standards are difficult to follow

Unit testing standards are difficult to follow when the code depends on slow operations, random values, time, databases, files, APIs, or external services. In this WeatherController task, the constructor, shutdown method, and hourly temperature query are intentionally slow, and the temperature values are randomly generated. This makes tests harder to keep fast and repeatable. A practical workaround is to move common setup into `@BeforeClass`, reuse the same test data, and close the controller once in `@AfterClass`. JUnit supports this style through lifecycle annotations, and Microsoft also recommends that unit tests should be fast, isolated, and repeatable.

Another difficult case is flaky testing, where the same test sometimes passes and sometimes fails without code changes. Google Testing Blog explains that flaky tests can be caused by concurrency, non-deterministic behaviour, third-party code, and infrastructure issues. The workaround is to remove randomness where possible, mock unstable dependencies, use fixed test data, and keep slow integration behaviour separate from unit tests.

References

Google Testing Blog. (2016). *Flaky tests at Google and how we mitigate them*.

JUnit. (2026). *JUnit documentation*.

Microsoft. (2025). *Unit testing best practices for .NET*.

Github Screenshot

